人工智能与深度学习

上机（二）

华中科技大学 机械科学与工程学院

陈伟

2026

PyTorch **关键步骤**

PyTorch **关键步骤**

- **整理输入输出数据，划分训练集与验证/测试集**

- **搭建神经网络**
  - 建立一个基于 nn.Module 的类
    - 在 __init__() 中定义需要用到的网络层
    - 在 forward() 中定义网络前向算法，即从网络输入计算输入的算法，forward() 的返回值是输出

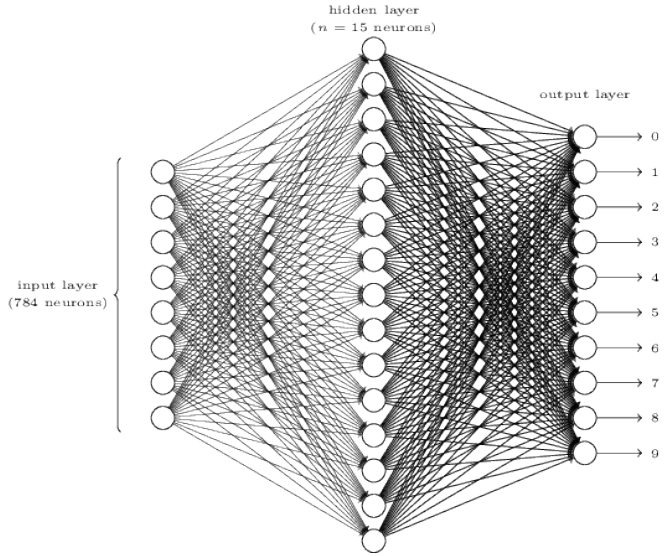
PyTorch 关键步骤

```
import torch
import torch.nn as nn

class NeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        self.fc1 = nn.Linear(784, 15)
        self.sigmoid = nn.Sigmoid()
        self.fc2 = nn.Linear(15, 10)

    def forward(self, x):
        out = self.fc1(x)
        out = self.sigmoid(out)
        out = self.fc2(out)
        return out
```

设计模型结构



PyTorch 关键步骤

- 选择计算硬件（GPU vs CPU）
- 建立模型实例
- 选择损失函数和优化器（包含优化器学习率参数）

```
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

model = NeuralNet().to(device)

# Loss and optimizer

criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)
```

PyTorch **关键步骤**

- **编写循环，训练神经网络**
  - 外层循环：回合（epoch）；一个回合包含了多个 mini-batch（内层循环），（通常）一个回合中所有的训练样本都会被用到
  - 内层循环：mini-batch；一个 mini-batch 中，执行一次随机梯度下降（即一次单步训练）

PyTorch 关键步骤

- **神经网络训练（单步，即单个 mini-batch）**
  - 由输入，通过神经网络模型，计算输出；计算损失函数
  - （通常）梯度清零；计算梯度；进行单步的梯度下降

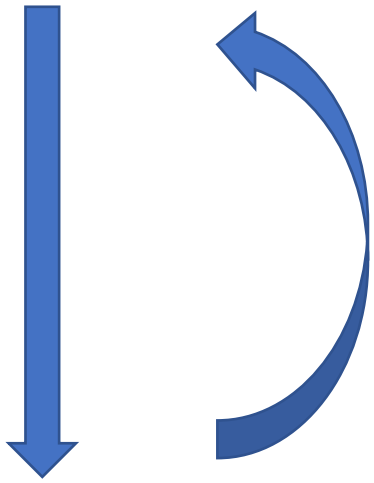
```
# mini-batch 的数据样本特征：images;  mini-batch 的数据样本标签：labels
# Forward pass

outputs = model(images)
loss = criterion(outputs, labels)

# Backward and optimize

optimizer.zero_grad() # 梯度清零
loss.backward()
optimizer.step()
```

下一个 mini-batch，循环进行



PyTorch 关键步骤

- 训练完成后，在测试或验证集上测试训练效果
- 根据需要，修改超参数、网络结构等，迭代改进

上机流程及报告要求

启动流程

- 进入 win10-x64 系统
- 在开始菜单中，找到 Anaconda，打开 Anaconda Navigator，打开 Jupyter Notebook

上机内容1

- **在 Jupyter Notebook 中依次运行以下 notebooks的每个单元格（即每个 cell），理解代码的含义**
  - MNIST_FC.ipynb
  - MNIST_CNN.ipynb

上机内容2

- **编写代码补充完成 Fashion_MNIST_incomplete.ipynb，并进行训练、调参**
  - 补充完成 class NeuralNet(nn.Module)
  - 设置并调整超参数，完成训练，争取在测试集上取得尽量好的效果

- **提交 .ipynb 文件和一份描述超参数调参过程及模型测试结果的报告**